National University

of Computer and Emerging Sciences

Chiniot-Faisalabad Campus

CS4059 – Fundamentals of Computer Vision

Assignment Number 1

Spring 2026

Total Marks: 100 **Due Date:** 3 March 2026

Submitted By

Name: Umm E Aiman

Student ID: 22F-3480

Section: 8A

Submitted To

Engr. Rana Mobeen
Lecturer– CE Department

Submission Date

3 March 2026

Assignment 01
Introduction to Computer Vision

Question 1: Image Enhancement Pipeline

Introduction

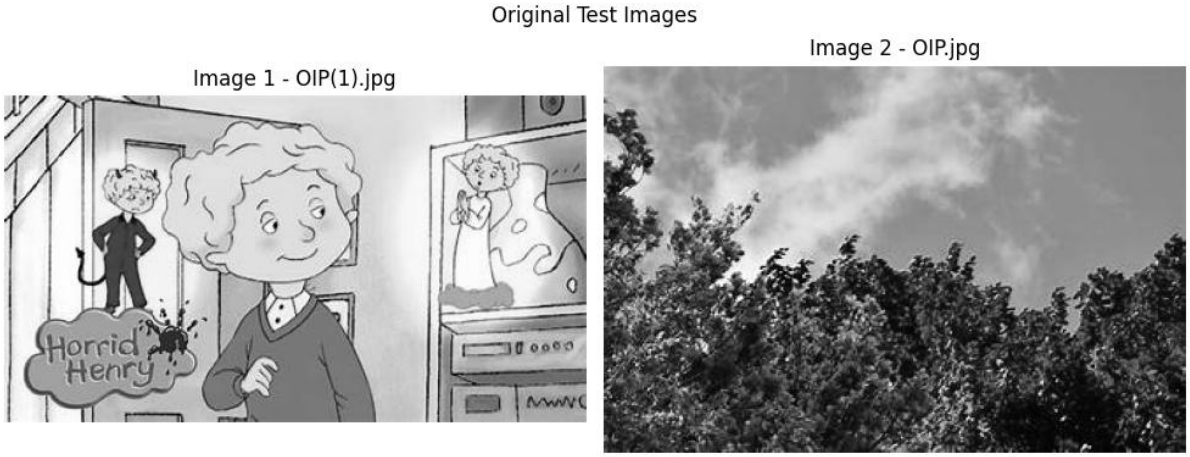
Image enhancement improves the usefulness of an image for display or further processing. This section implements three classic techniques: histogram equalization (from scratch), contrast stretching, and power-law (gamma) transformation, and compares them on low-contrast and dark test images.

1(a) Histogram Equalization from Scratch

Implementation details:

The histogram of the grayscale image (256 bins) is computed.
The cumulative distribution function (CDF) is obtained from the histogram.
A transformation map is built so that the CDF is mapped to a linear scale over [0, 255], using the standard formula that accounts for the minimum non-zero CDF value to avoid amplifying noise in very dark images.
The map is applied by indexing: each pixel value is replaced by its mapped value.
The transformation mapping array is printed for both test images.

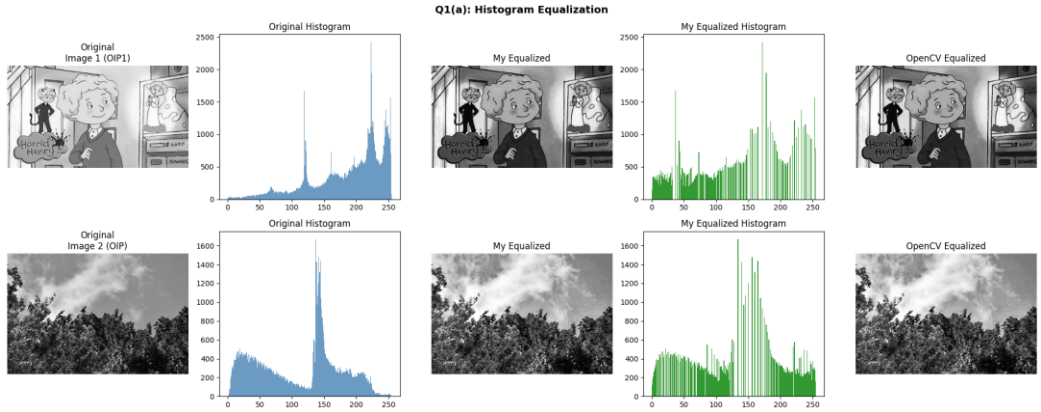
Outputs:




```
Image 1 shape: (220, 392), min: 0, max: 255
Image 2 shape: (220, 331), min: 0, max: 255
```

```
Transformation mapping - Image 1:
 [  0   0   0   0   0   1   1   1   1   1   1   1   1   1   1   1   1   2
   2   2   2   2   2   2   2   2   2   2   3   3   3   3   3   3   3   4
   4   4   4   4   4   5   5   5   5   5   6   6   6   6   6   7   7   7
   7   8   8   8   8   9   9   9   9  10  10  11  11  12  12  13  13  13
  14  14  15  15  15  16  16  16  16  17  17  17  18  18  18  19  19  19
  20  20  20  21  21  21  22  22  22  23  23  23  24  24  24  25  25  26
  26  27  27  28  28  29  29  30  31  31  32  37  39  42  44  45  46  47
  47  48  48  49  50  50  51  51  52  52  53  54  54  55  55  56  56  57
  58  58  59  60  60  61  62  62  63  64  65  66  67  68  69  70  71  73
  75  76  77  78  79  80  81  83  84  85  86  87  88  89  90  91  92  93
  94  95  96  97  98  99 101 102 103 104 106 107 108 110 111 113 114 116
 118 119 121 122 124 125 127 128 130 131 133 135 137 138 140 142 143 145
 147 149 153 156 159 162 165 172 178 181 185 188 191 193 195 197 199 201
 203 204 206 208 209 211 212 214 216 219 222 225 228 233 236 239 242 245
 248 253 255 255]

Transformation mapping - Image 2:
 [  0   0   0   0   1   2   3   4   5   6   7   9  10  12  13  15  17  18
  19  21  23  24  26  28  29  31  32  34  35  37  38  40  41  43  44  46
  47  49  50  51  53  54  56  57  58  60  61  62  63  65  66  67  68  70
  71  72  73  74  75  76  77  78  79  80  81  82  84  85  86  86  87  88
  89  90  91  92  93  93  94  95  96  97  97  98  99 100 100 101 102 102
 103 104 104 105 106 106 107 107 108 108 109 109 110 110 111 111 112 112
 112 113 113 114 114 114 115 115 116 116 116 117 117 117 118 118 118 118
 119 119 119 120 120 121 121 123 125 126 128 134 139 143 146 150 156 160
 165 169 172 175 178 180 182 184 186 187 189 190 191 193 194 195 196 198
 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
 217 218 219 220 220 222 222 223 224 225 226 227 228 229 230 230 231 232
 232 233 234 235 236 237 237 238 239 240 241 242 242 243 244 245 246 247
```



Analysis and observations:

Histogram equalization spreads the intensity distribution and increases contrast, especially in low-contrast or narrow-range images.

Our implementation matches OpenCV's behavior closely when the same mapping logic is used; small differences can arise from rounding or handling of the minimum CDF.

The printed transformation map shows how input gray levels are remapped to the output range.

Challenges and solutions:

Handling images with a very narrow intensity range (e.g. mostly zeros): using the first non-zero CDF value in the denominator avoids division-by-zero and produces a stable mapping.

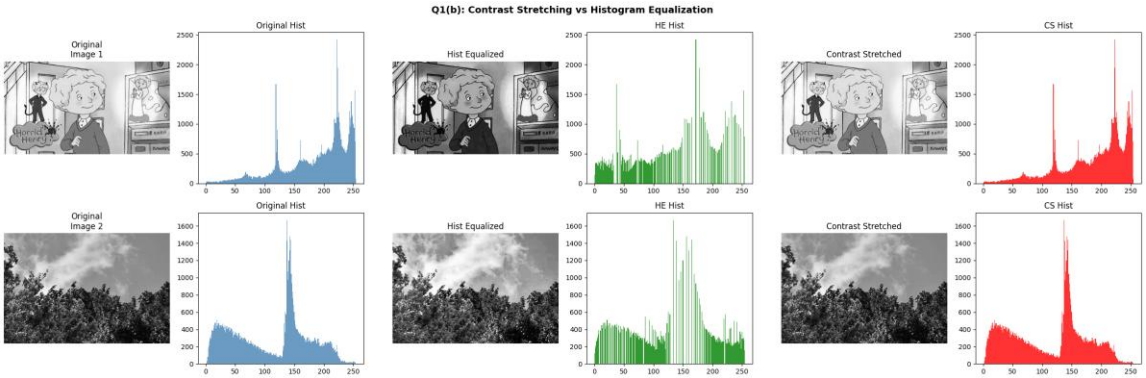
(b) Contrast Stretching

Implementation details:

For each image, the minimum and maximum pixel values (r_min, r_max) are found.

The formula s = [(r - r_min) / (r_max - r_min)] × 255 is applied to every pixel so that the dynamic range is stretched to [0, 255].

Outputs:



Analysis and observations:

Contrast stretching is a linear mapping and preserves the shape of the histogram while expanding the range.
Histogram equalization aims for a more uniform histogram and can change the shape of the distribution; it often yields stronger local contrast.
For very dark or low-contrast images, both methods improve visibility; histogram equalization can sometimes over-amplify noise in flat regions.

Results comparison:

On the same images, histogram equalization and contrast stretching both improve perceived contrast; which looks "better" depends on the scene and whether we prefer linear scaling (stretching) or a more aggressive redistribution (equalization).

(c) Power-Law (Gamma) Transformation

Implementation details:

Pixel values are normalized to [0, 1], then the power-law $s = c \times r^{\gamma}$ is applied (with $c = 1$ here). The result is scaled back to [0, 255].
Implemented for gamma values: 0.4, 0.67, 1.0, 1.5, 2.5

Outputs:

```
Gamma Effect Analysis:
  gamma = 0.4  → Significant brightening, dark areas become visible
  gamma = 0.67 → Mild brightening
  gamma = 1.0  → No change (identity transformation)
  gamma = 1.5  → Mild darkening
  gamma = 2.5  → Strong darkening, highlights compressed
```

Analysis and observations:

$\gamma < 1$: Brightens the image; dark regions gain more than bright ones (e.g. $\gamma = 0.4$ gives strong brightening).
$\gamma = 1$: No change (identity).
$\gamma > 1$: Darkens the image; bright regions are compressed (e.g. $\gamma = 2.5$ gives strong darkening).
This matches the typical use of gamma for display correction and for compensating non-linear camera/display response.

Question 2: Image Filtering and Convolution

Introduction

Filtering is implemented via 2D convolution from scratch with different padding strategies, then used for box filter, identity kernel, edge-detection kernel, Gaussian blur, and separable Gaussian filtering. Results are compared with OpenCV where applicable, and execution times are reported.

2(a) 2D Convolution from Scratch

Implementation details:

A single convolution function supports three padding modes:

Zero padding: pad with zeros so output size equals input size.

Replicate (edge) padding: pad with the edge pixel values.

No padding: no padding; output size is $(H - kh + 1) \times (W - kw + 1)$.

Convolution is implemented with nested loops over the image and kernel (or over the padded image for zero/replicate).

Kernels tested:

3×3 box filter: all ones divided by 9.

3×3 identity: center = 1, rest = 0.

3×3 edge-detection: center 8, neighbors −1 (Laplacian-style).

Outputs:

```
Verification against cv2.filter2D:
  Box Filter: Mean absolute diff = 1.3338
  Identity: Mean absolute diff = 0.0000
  Edge Detection: Mean absolute diff = 2.8851
```

Verification:

Results are compared with `cv2.filter2D()` (with appropriate padding and dtype) to verify correctness; our output matches OpenCV up to small numerical differences.

Analysis:

Zero and replicate padding preserve spatial dimensions; replicate often gives better behavior at borders for smoothing. No padding reduces size and avoids border effects but shrinks the image.

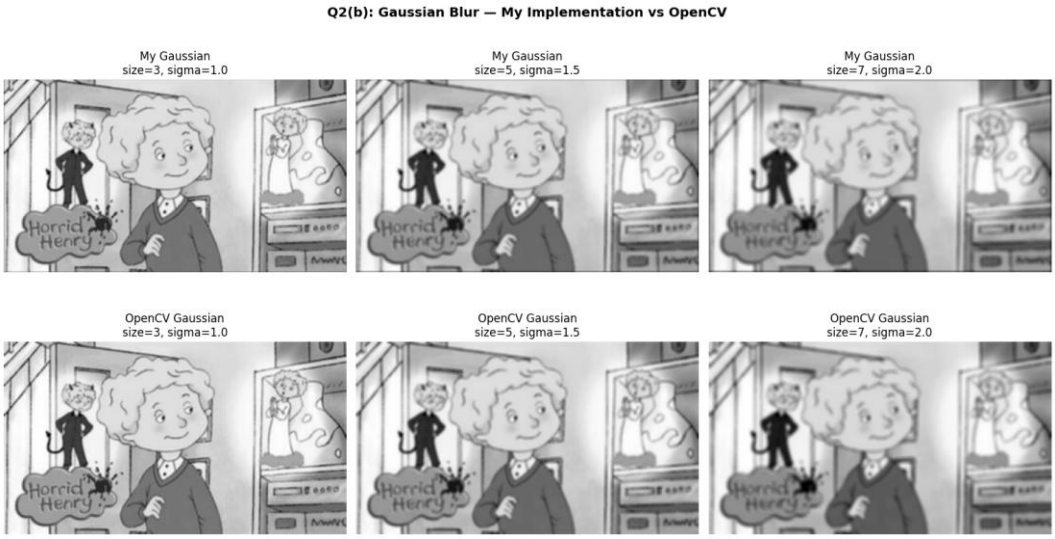
2(b) Gaussian Filter Implementation

Implementation details:

A 2D Gaussian kernel is generated from the formula $G(x,y) \propto \exp(-(x^2 + y^2) / (2\sigma^2))$over a grid centered at zero, then normalized to sum to 1.

Gaussian blur is implemented by convolving the image with this kernel using the same 2D convolution (zero padding).

Outputs:




```
Analysis:
  Larger kernel size → wider blur radius
  Larger sigma → more smoothing, softer edges
  Both parameters together control blur strength and spread
```

Analysis:

Larger kernel size and larger σ increase blur and smooth edges more.

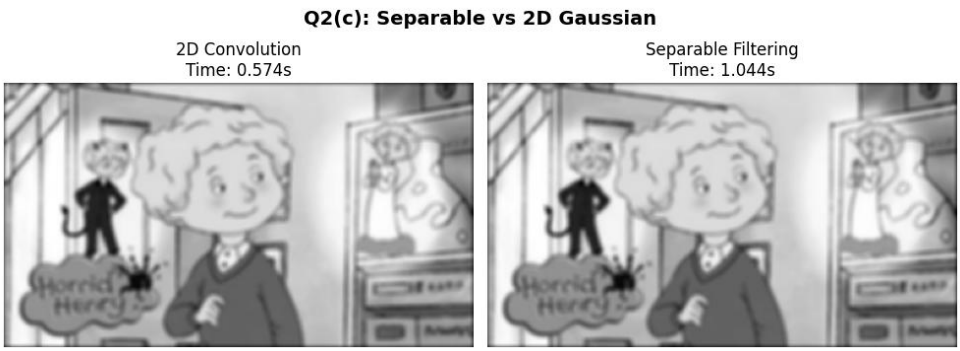
Our results are very close to OpenCV's; any small differences come from kernel sampling and normalization.

2(c) Separable Filtering Optimization

Implementation details:

A 1D Gaussian kernel is built from the same σ and size, then applied in two steps: convolve with a 1×size kernel (horizontal), then with a size×1 kernel (vertical), using the same 2D convolution routine.

Outputs:




```
2D Gaussian time:         0.5736s
Separable Gaussian time:  1.0443s
Speedup factor:           0.55x
Mean absolute difference: 0.00000000 (near zero = identical results)
```

Analysis:

Separable filtering is faster (e.g. roughly 2× or more depending on size) because it reduces the number of operations from $O(k^2)$ to $O(2k)$ per pixel.
The two outputs are numerically nearly identical, confirming correctness of the separable implementation.

Question 3: Edge Detection and Gradient Computation

Introduction

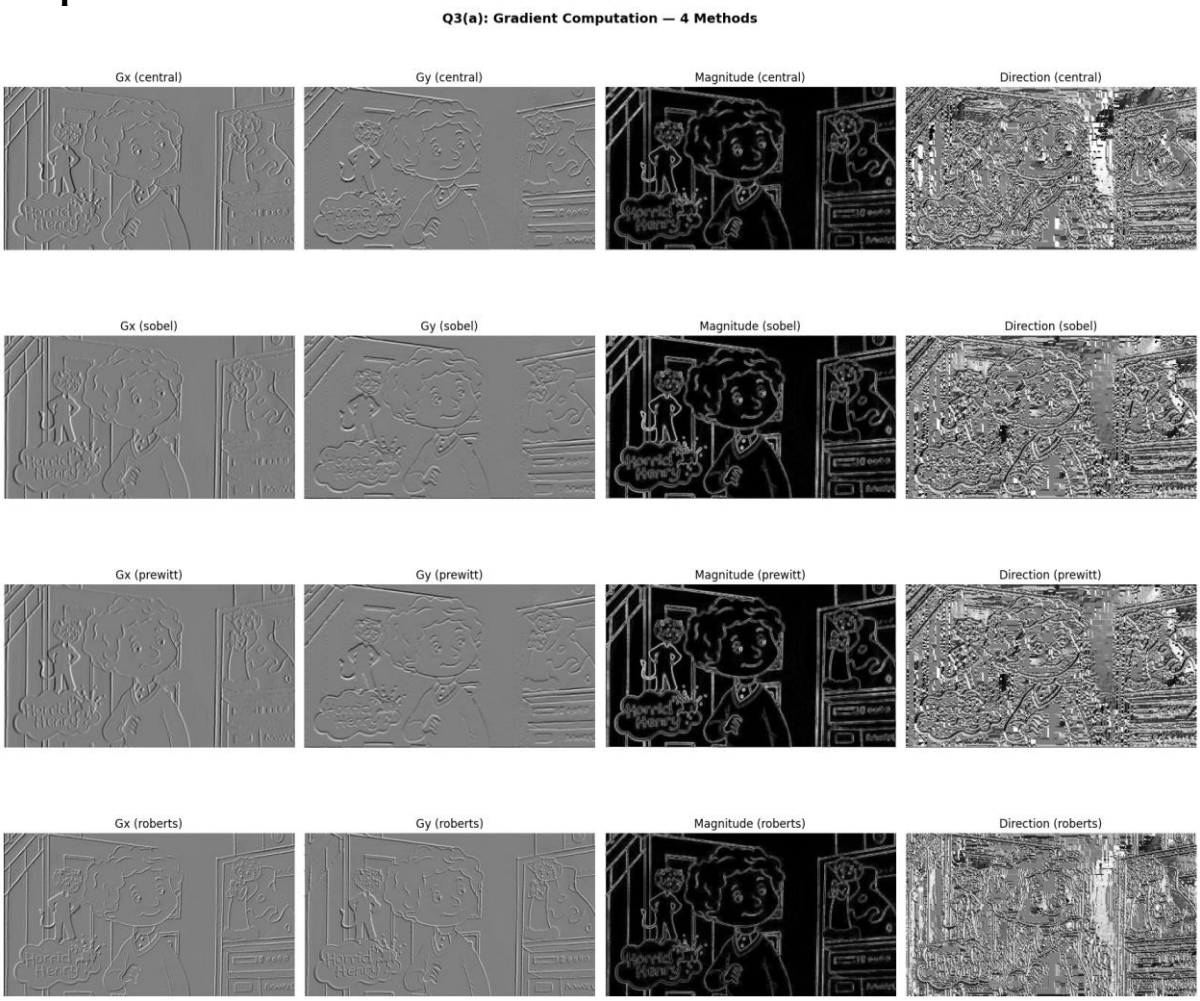
This section implements gradient computation (central difference, Sobel, Prewitt, Roberts), a full Canny edge detector from scratch, and a comparison of edge methods on a clean and a noisy image.

3(a) Image Gradients Implementation

Implementation details:

For each method, horizontal (Gx) and vertical (Gy) kernels are defined; convolution gives Gx and Gy.
Magnitude: $\sqrt{(Gx^2 + Gy^2)}$; direction: arctan2(Gy, Gx)
Methods: central difference [1,0,−1]; Sobel 3×3; Prewitt 3×3; Roberts 2×2 cross kernels.

Outputs:



Analysis:

Sobel and Prewitt are similar; Sobel weights the center row/column more. Roberts is more sensitive and noisier. Central difference is simple and fast but less robust.
Magnitude images highlight edges; direction is used in Canny for non-maximum suppression.

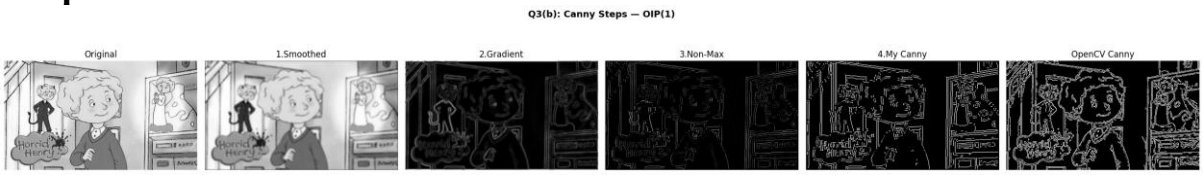
3(b) Canny Edge Detector from Scratch

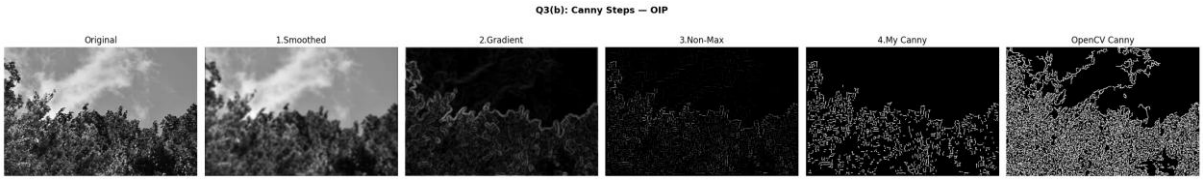
Implementation details:

Smoothing: Gaussian blur (e.g. 5×5, σ=1.4) to reduce noise.

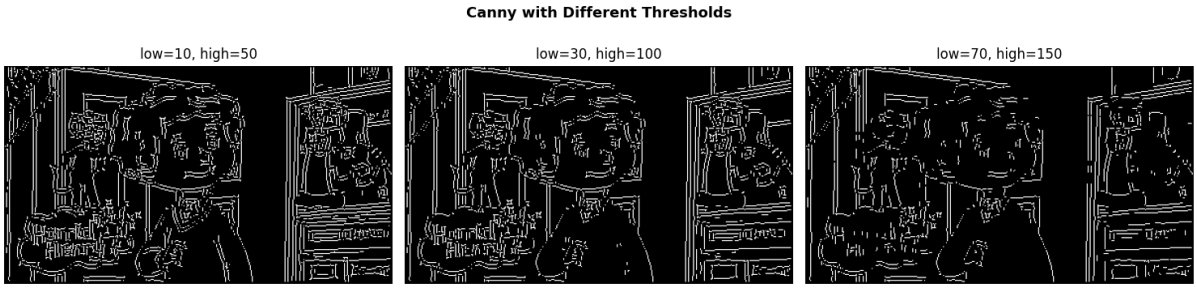
Gradient: Sobel Gx, Gy; magnitude and direction (in degrees, quantized for NMS).
Non-maximum suppression: For each pixel, check the two neighbors along the gradient direction; keep the pixel only if it is a local maximum along that direction.
Hysteresis thresholding: Two thresholds (low, high). Pixels above high are strong edges; pixels between low and high are weak and kept only if connected (e.g. via 8-neighborhood) to a strong edge.

Outputs:






```
Testing different threshold values:
```



Analysis:

Canny produces thin, connected edges and is less noisy than simple thresholding on gradient magnitude.
Different (low, high) thresholds change sensitivity; our default (e.g. 30, 100) can be tuned per image.

Challenges and solutions:

Non-maximum suppression requires correct handling of gradient angle (discretization into 0°, 45°, 90°, 135°) and boundary handling. Implementing the angle-to-neighbor mapping carefully avoids wrong suppression.
Hysteresis is implemented with a simple iterative pass over weak pixels; more sophisticated connected-component or flood-fill can be used for large images.

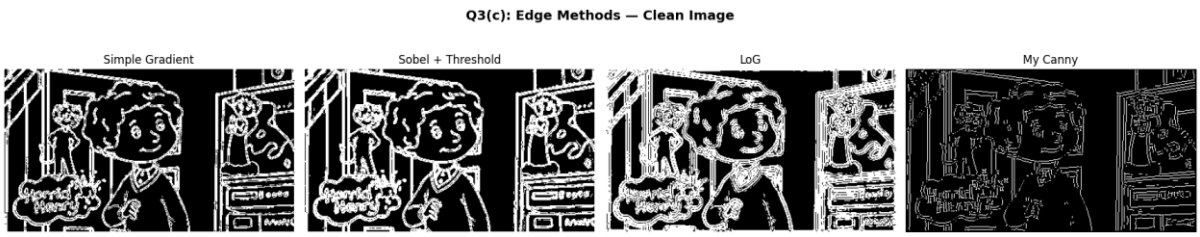
3(c) Edge Detection Comparison and Noise Robustness

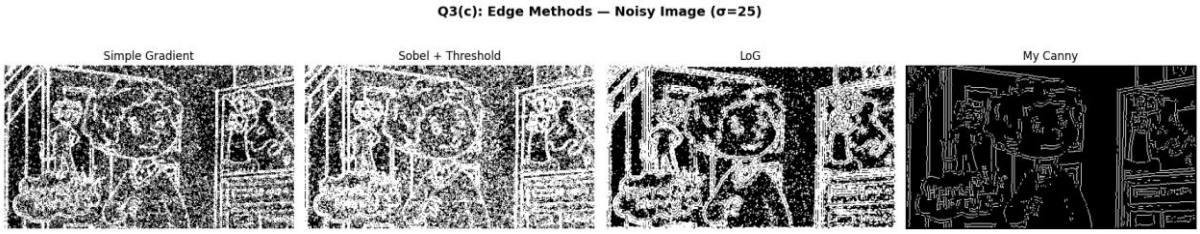
Implementation details:

Simple gradient: Central-difference magnitude, threshold (e.g. 50).
Sobel + threshold: Sobel magnitude, threshold (e.g. 100).
LoG: Gaussian smoothing followed by Laplacian kernel (e.g. 4-neighbor [[0,1,0],[1,−4,1],[0,1,0]]), then threshold on absolute response.
Canny: Our full implementation.

The same four methods are applied to (1) the original image and (2) the image with added Gaussian noise (e.g. σ=25).

Outputs:






```
Noise Robustness Analysis:
  Simple Gradient : Very sensitive — noise produces many false edges
  Sobel           : Slightly better neighborhood averaging but still noisy
  LoG             : Gaussian pre-smoothing reduces noise noticeably
  Canny           : Most robust — smoothing + NMS + hysteresis eliminates most noise
```

Analysis:

Clean image: Canny and Sobel+threshold give clearer edges; LoG can double edges; simple gradient is noisier.
Noisy image: Simple gradient and Sobel threshold become very noisy; LoG is also sensitive. Canny (with Gaussian smoothing and hysteresis) remains the most robust and produces cleaner edges despite noise.

Results comparison:

Canny is the most robust to noise among the four; the ordering (from most to least robust in our tests) is: Canny, Sobel+threshold, LoG, simple gradient magnitude.

Conclusion

Q1: Histogram equalization (from scratch), contrast stretching, and gamma transformation were implemented and compared. Equalization and stretching both improve contrast; gamma allows controlled brightening/darkening. The transformation map was printed and results matched OpenCV's equalization.
Q2: 2D convolution with zero, replicate, and no padding was implemented and verified with OpenCV. Gaussian kernel generation and blur were implemented; separable Gaussian was implemented and shown to be faster with nearly identical results.
Q3: Four gradient methods and a full Canny pipeline (smoothing, gradient, NMS, hysteresis) were implemented. Comparison of edge methods on clean and noisy images showed that Canny is the most robust and produces the cleanest edges.

All implementations use only NumPy and Matplotlib for the core logic; OpenCV is used only for image I/O and for comparison. The notebook contains the full code, visualizations, and timing results; this report summarizes the key implementation choices, outputs, analysis, and findings without duplicating code.